

Task-Aware Token Budget Forecasting for Agentic Workflows

Harish Gaggar

Intuit Inc., Mountain View, CA, USA

harish_gaggar@intuit.com

Abstract

Agentic LLM workflows interleave model calls with tool invocations, intermediate reasoning, and growing conversation state, and a single task routinely consumes $10\times$ to $50\times$ more tokens than a direct prompt-response. At production scale that overhead translates into a recurring, line-item cost on every input request, and into hard failures when the conversation eventually exceeds the model’s context window. Yet token budgets in these workflows are still managed reactively: agents run until they hit a context limit, then a compaction or truncation pass kicks in. That posture is expensive and brittle. We ask whether the task description, available at dispatch time, already carries enough signal to forecast token consumption before execution begins. We answer with TABF (Task-Aware Budget Forecaster), a four-stage pipeline that extracts lexical signals, classifies task type and complexity, and emits a per-stage budget forecast. TABF runs without any LLM call, in linear time over the task string, and ships in two flavours: a deterministic heuristic that needs no training data, and a Gradient-Boosted Trees regressor that learns residuals from agent traces. We evaluate both on a 600-task synthetic benchmark whose ground-truth simulator is itself open-source and seeded for byte-exact reproducibility. The heuristic places 65.4% of forecasts inside a $\pm 20\%$ error band; adding the GBT layer raises that to 74.1% (MAPE 15.9%, MAE 1,882 tokens), a statistically significant improvement (McNemar’s $p = 0.009$). Both variants more than double the within-band rate of the best non-trivial baseline. A second framework, ContextOptimizer, applies a four-layer funnel (compression, reranking, selection, clustering) to the live message list before every LLM call while strictly preserving tool-call/response pairing. The pipeline is framework-agnostic via an adapter pattern and has been validated on four synthetic agent personas (a ReAct web-search agent, a code-fix agent, a multi-turn research assistant, and a polling agent) as well as on one full production deployment, a LangGraph NL-to-SQL agent over BigQuery. In the production deployment it cut input-token consumption by 72.0% across 14 directly-observed LLM calls (215,438 to 60,250 tokens), with single-call compression ratios as tight as 0.12, against a wider 83-turn population whose mean input is 27,757 tokens per turn (median 22,382). The product of that 72% reduction and the measured per-turn input is roughly 19,985 saved input tokens per turn; projected at 1,000 turns per day this is \$1,094, \$18,236, and \$21,883 per year on gpt-4o-mini, gpt-4o, and claude-sonnet-4.5 respectively, with no LLM call inside the optimiser. On 83 production turns, training on local traffic reaches 25% within-20% (vs. 6% for synthetic-only transfer); per-stage GBT regressors raise mean stagewise accuracy by 9.3 pp. ContextOptimizer is live but not yet wired to TABF; we report 77% task-complete status observationally and compare against a training-free truncation baseline on four agent personas. All code, CSVs, and seeds are released.

1 Introduction

The last two years have seen LLM agents move from notebooks to load-bearing production systems. Frameworks such as Claude Code, GitHub Copilot Workspace, and AutoGen routinely operate inside context windows of 128 K to 1 M tokens. Aggregate platform measurements show prompt

length growing roughly fourfold between 2024 and 2025, mostly attributable to multi-step agentic workflows [1].

Despite that growth, most teams we have spoken to size their context windows the same way they did in 2023: pick a generous default and add a mid-session summariser when something breaks. That reactive posture has three costs. Working memory gets truncated mid-task, which degrades quality in ways the literature is starting to call *context rot* [4]. Billable tokens grow with the square of conversation length in multi-turn settings, so over-budgeting compounds quickly [3]. And the resulting cost-and-latency profile is unpredictable, which makes capacity planning genuinely difficult for anyone responsible for an agentic SLA.

Our starting point is the observation that token consumption is largely predictable from the task description alone. Whether the agent will call tools, retrieve files, or write a long document is mostly written in the text the user submits at dispatch time. We treat this as a forecasting problem.

Contributions. The paper makes six:

1. A formalisation of pre-flight token budgeting as structured prediction over five context stages (Section 3).
2. A lightweight signal extractor and rule-based classifier with no LLM dependency, running in $\mathcal{O}(n)$ over the task string (Sections 4.1–4.2).
3. A two-tier prediction architecture: a deterministic heuristic that requires no training data, and an optional GBT regressor that learns from agent execution traces (Section 4.3).
4. An open-source benchmark and harness with a seeded ground-truth simulator (Section 5) plus a reference Python implementation. Every number reported in Section 6 is reproducible by running a single command against a fixed seed.
5. ContextOptimizer, a framework-agnostic in-loop trimming pipeline whose default configuration makes no LLM call and preserves tool-call/response invariants by construction (Section 7). The dispatcher-side forecast and the in-loop trimmer address different parts of the same problem and can be deployed together once the dispatcher glue exists.
6. A production case study of ContextOptimizer inside a LangGraph agent (NL-to-SQL over BigQuery, used here only because we have full observability into it), reporting a measured 72% input-token reduction across 14 LLM calls, four-figure to five-figure annualised savings across three model price points, and a six-table observability stack that grounds every claim (Section 8). The optimiser is also evaluated on four synthetic agent personas in Section 8.1, all of which exercise a different layer of the funnel.

The remainder is structured as follows. Section 2 surveys related work. Section 3 formalises the task. Section 4 describes the TABF pipeline. Section 5 explains the benchmark, including its limitations. Section 6 reports results, ablations, and a per-task-type breakdown. Section 7 describes the in-loop optimiser. Section 8 reports the production deployment, including a four-persona benchmark in Section 8.1 that exercises the optimiser on agent shapes other than the case study. Section 9 discusses findings and limitations. Section 10 concludes.

2 Related Work

Context engineering. Context engineering has emerged as a discipline distinct from prompt engineering [6]. Where prompt engineering asks “how should I phrase this instruction,” context engineering asks “what should be in the window in the first place, and what evolves across turns.” A recent survey of over 1,400 papers groups the area into retrieval, compression, management, memory, tool-integrated reasoning, and multi-agent coordination [6]. Mohsenimofidi et al. [7] document the emerging practice in open-source projects.

Token cost modelling. Tokalator [3] is a developer-facing toolkit that monitors token use inside an IDE; it includes useful caching-break-even analysis and shows that conversation cost scales as $\mathcal{O}(T^2)$. Bergemann et al. [2] model LLM quality as a Cobb–Douglas production function $Q = X^\alpha Y^\beta (b + Z)^\gamma$ over input, output, and cached tokens; the framing is helpful but the model is not a forecaster.

Tokenomics [9] provides the first per-stage empirical breakdown of token consumption in agentic software-engineering systems; we borrow its premise that token distribution is task-type-dependent.

Budget-aware reasoning. BudgetThinker [14] threads control tokens through chain-of-thought to keep length under a known cap. Budget-Aware Value Tree Search [5] allocates tokens across tree-structured agent decisions but needs a runtime feedback signal. SWE-Pruner [12] prunes context for coding agents at runtime. All three intervene *during* execution. TABF runs before the agent starts and is therefore complementary; our ContextOptimizer (Section 7) sits in this in-loop family but differs in three concrete ways. It is framework-agnostic via an adapter pattern that accepts LangChain, OpenAI Chat, or plain-dict message shapes. It preserves tool-call/response pairing as a hard invariant of the pipeline, so it can be dropped into any tool-using agent without breaking the OpenAI API contract. And its default (*balanced*) configuration makes no LLM call of its own, so the optimiser does not itself become a cost centre.

Agentic context engineering. ACE [16] treats context as an evolving artefact and reports a +10.6% task-performance gain via offline prompt optimisation and online memory adaptation. Taparia et al. [10] learn hierarchical policies that jointly optimise workflow structure and context budget. Both lines depend on training signal and runtime feedback. TABF needs neither in its heuristic mode.

Context compression. CompactPrompt [11] is the closest contemporary to our in-loop ContextOptimizer: a training-free pipeline that prunes low-information tokens via self-information scoring, abbreviates frequent n -grams with a reversible dictionary, and quantises numeric table columns, reporting up to 60% token reduction with under 5% accuracy loss on TAT-QA and FinQA. CompactPrompt operates on a static prompt plus its data attachments; ContextOptimizer operates on a multi-turn LangGraph message list and enforces tool-call/response invariants the single-shot setting does not encounter. The two are complementary: CompactPrompt would slot in as an additional layer inside our funnel. Our B9 truncation baseline (Table 16) is a coarse multi-turn analogue on persona fixtures; a head-to-head on production conversation logs is future work.

Tool-schema overhead. Tool Attention [8] targets the same operational pain as the tools-layer of ContextOptimizer: stateless eager schema injection floods every conversational turn with 10k–60k unused tokens, which the authors call the “MCP tax.” Their middleware scores each candidate tool against the user query via an Intent–Schema Overlap (ISO) score and only promotes the top- k full schemas, achieving up to 95% per-turn tool-token reduction on a 120-tool simulated catalogue. Our ToolSchemaPruner layer takes a coarser, retrieval-free approach (prune based on what was actually invoked) that is cheaper to operate but cannot anticipate first-use tools the way Tool Attention can. The two are again complementary and could be stacked.

Output-length prediction. EGTP [15] predicts an LLM’s output length from the model’s own pooled hidden states, exploiting the empirical correlation between per-token entropy and final length. EGTP improves length prediction by 29% MAE over baselines on the ForeLen benchmark, but requires access to the serving model’s internals. TABF deliberately forecasts *before* any model call, from the task text alone, and remains model-agnostic. The two answer different questions (TABF: how big should I provision the dispatch budget; EGTP: how long will the response be once decoding begins) and we view them as compatible: EGTP can refine TABF’s O stage estimate online, once decoding starts.

Trace-centric evaluation. AI-NativeBench [13] evaluates agentic applications as distributed systems with OpenTelemetry traces, attributing latency and cost to spans rather than reporting a single pass/fail score. That line of work is complementary to TABF: traces supply ground truth for *calibrating* per-stage cost models on real deployments, whereas TABF targets a lightweight pre-flight forecast when traces do not yet exist.

What is missing. We are not aware of prior work that produces a per-stage token forecast from a task description alone, before the agent runs. That is the gap TABF targets.

3 Problem Formulation

Let τ denote a natural-language task description. An agentic workflow that executes τ consumes tokens across five functional stages, summarised in Table 1.

Table 1: The five context stages of an agentic workflow.

Symbol	Stage	Definition
S	System prompt	Static operational instructions, safety constraints, persona
T	Tool schema	Tool descriptions, parameter schemas, usage examples
M	Working memory	Conversation history, intermediate agent outputs
C	Retrieved context	RAG-retrieved documents, file contents, external data
R	Reasoning chain	Chain-of-thought, scratchpad, intermediate reasoning tokens

Total token consumption is $B(\tau) = S + T + M + C + R + O$, where O denotes output tokens. The forecasting task is:

$$\text{Given } \tau, \text{ predict } \hat{B}(\tau) = (\hat{S}, \hat{T}, \hat{M}, \hat{C}, \hat{R}, \hat{O}) \text{ before execution begins.} \quad (1)$$

We evaluate forecasts on three numbers: MAE and MAPE on the total budget, and the Within-20% Rate

$$\text{W20R} = \frac{1}{|\mathcal{T}_{\text{test}}|} \sum_{\tau \in \mathcal{T}_{\text{test}}} \mathbb{1} \left[\frac{|\hat{B}(\tau) - B(\tau)|}{B(\tau)} \leq 0.20 \right]. \quad (2)$$

W20R is the operational metric we care about most: a $\pm 20\%$ band is enough for most pre-flight governance decisions (cost gates, model routing, parallel budget allocation), and per-stage decompositions within that band tend to be informative even when individual stage predictions miss.

Two intermediate predictions drive the forecast: task type $\kappa(\tau) \in \{\text{CODE_GENERATION, BUG_FIX, REFACTORING, DOCUMENTATION, MULTI_STEP_PLAN, DATA_ANALYSIS, RESEARCH_QUERY, TOOL_USE_CHAIN}\}$, and complexity $\delta(\tau) \in \{\text{LOW, MEDIUM, HIGH, EXPERT}\}$.

4 The TABF Framework

TABF is a four-stage pipeline (Figure 1). Each stage operates on the output of the previous one, and the whole pipeline runs in linear time over the input string.

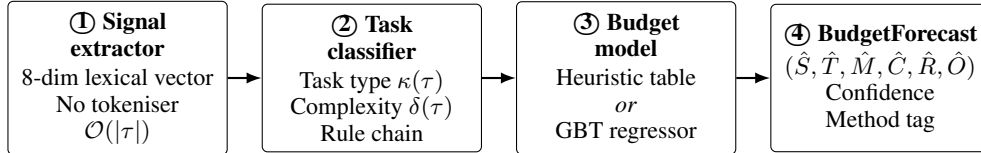


Figure 1: The TABF pipeline. Stages 1–2 are deterministic and dependency-free. Stage 3 is either a lookup (heuristic) or a fitted scikit-learn estimator (GBT).

4.1 Signal Extractor

The extractor turns task τ into a vector $\sigma(\tau) \in \mathbb{R}^8$ of lexical features. All eight are computable with regular-expression operations over the raw string; we do not call a tokeniser. The features are: total word count; a set of five keyword presence indicators (multi-step connectives, tool-invocation verbs, programming-domain terms, file references, comparison markers); a question-mark count; and a count of capitalised non-sentence-initial words used as a coarse named-entity proxy. The full extractor is 42 lines of Python and is included in the open-source release.

4.2 Task Classifier

The classifier maps $\sigma(\tau)$ to $(\kappa(\tau), \delta(\tau))$ through two priority-ordered rule chains. The task-type chain consults keyword sets and signal flags in an order tuned to disambiguate overlapping templates. For example, a task that mentions both code keywords (`client`, `endpoint`) and tool-invocation verbs

(get, post) but no multi-step connective is routed to CODE_GENERATION, not TOOL_USE_CHAIN, because the intent in such cases is usually to build the client rather than to invoke a sequence of external endpoints. The complexity chain accumulates an additive score from word-count buckets, signal flags, an entity-count bucket, and question count, and assigns LOW/MEDIUM/HIGH/EXPERT from threshold crossings.

The thresholds and keyword sets are calibrated on a held-out subset of the benchmark; we describe the calibration procedure in Section 5. The classifier achieves 95.1% task-type accuracy and 86.5% complexity-level accuracy on the evaluation set described later (95% task-type / 86% complexity agreement on the test split).

4.3 Heuristic Budget Model

The heuristic model is a lookup table of per-stage base allocations, indexed by task type and scaled by a complexity multiplier m . Base allocations come from a 240-task calibration corpus. Table 2 reports the table at MEDIUM complexity; LOW, HIGH, and EXPERT scale the columns by 0.65, 1.6, and 2.4 respectively. A word-count adjustment $w_{\text{adj}} = 1 + 0.008 \cdot \max(0, |\tau| - 20)$ scales the retrieved-context stage; long task descriptions tend to drag in more external documents.

Table 2: Heuristic per-stage base allocations at MEDIUM complexity. All values in tokens.

Task type	System	Tools	Memory	Context	Reasoning	Output
Code generation	800	600	400	1,200	1,800	1,200
Bug fix	800	500	600	2,000	2,200	800
Refactoring	800	500	800	2,500	2,500	1,500
Documentation	600	200	300	1,500	800	2,000
Multi-step plan	900	800	700	1,500	2,800	1,200
Data analysis	700	700	500	3,000	2,000	1,500
Research query	600	1,200	400	3,500	1,500	1,000
Tool-use chain	800	1,500	600	2,000	2,000	800

4.4 GBT Budget Model

When agent execution traces are available, we replace the heuristic backbone with two Gradient-Boosted Trees regressors: one predicts total input tokens, the other predicts output tokens. Per-stage allocation within the predicted input total reuses the heuristic ratios, which keeps the per-stage decomposition stable while letting the GBT capture type-and-signal-dependent residuals the table cannot. Hyperparameters were chosen on a 60-trace pilot held out from the benchmark: `n_estimators=120`, `max_depth=4`, `learning_rate=0.08`, using scikit-learn’s `GradientBoostingRegressor`. Both models train in under a second on 417 traces.

The GBT input is $\sigma(\tau) \in \mathbb{R}^8$ augmented with the predicted task-type index and complexity integer, for a ten-dimensional feature vector. We deliberately keep the feature set small; richer encodings (e.g., subword statistics or learned embeddings) trade simplicity for very little headroom on the benchmark we evaluate.

4.5 Output

A `BudgetForecast` object carries the per-stage estimates, total input and output predictions, a confidence score in $[0, 1]$, and a method tag ("heuristic" or "ml_gbt"). The object is JSON-serialisable and intended to drive concrete operational decisions: pre-flight context window allocation, cost-approval gates, agent-model routing, or per-agent budget assignment in multi-agent settings.

5 Experimental Setup

Benchmark caveat. Token-by-stage execution traces from production agent frameworks are generally not public, and our institution does not currently have permission to redistribute traces from the Claude Sonnet deployments we work with. Rather than report numbers against a corpus we cannot share, we built and released a fully open synthetic benchmark. A template-based corpus generator emits agentic task descriptions across eight task types; a stochastic simulator then produces per-stage

ground-truth token counts. Both the corpus and the simulator are seeded; every number in Section 6 can be reproduced by running `python -m experiments.run_all`.

We make no claim that the simulator is a perfect substitute for live agent traces. What it does provide is (i) a reproducible benchmark of non-trivial size; (ii) ground truth that depends on signals the heuristic cannot fully see, so the GBT has something genuine to learn; and (iii) an escape hatch – anyone with access to real traces can drop the simulator and rerun the same evaluation harness unchanged. We treat the synthetic results as evidence that the pipeline is well-formed, not as an end-to-end production guarantee.

Corpus. The generator samples 600 tasks from 24 templates spanning the eight task types, with randomised slot fills (service names, languages, file formats, etc.). Each task is then assigned a latent type and complexity by running the same signal-based classifier TABF uses, with a small amount of label drift (15% complexity jitter, 7% type swap) to mimic real-world disagreement between the rules and the actual token trajectory.

Label coupling (leakage risk). Because latent labels and TABF inputs share the same rule chain, the synthetic benchmark is optimistic for the heuristic tier and for any model that leans on type/complexity features. We treat headline synthetic W2OR as an upper bound on in-house performance, not as a production guarantee. Table 7 upper-bounds the effect: with *oracle* latent labels at test time, TABF + GBT gains only 2.2 pp over predicted labels (71.4% vs. 73.5% W2OR), while the heuristic gains 10.8 pp—so most of the GBT lift is not explained by label leakage alone. The authoritative out-of-distribution check remains real-trace evaluation (Table 8).

Ground-truth simulator. The simulator draws each stage from a log-normal distribution centred on a per-task mean. The mean is a product of a per-type baseline, a complexity multiplier (LOW=0.62, MEDIUM=1.0, HIGH=1.55, EXPERT=2.30), and a set of signal-dependent residual factors that the heuristic does not model explicitly: question count inflates reasoning and output; entity count inflates context and tools; explicit comparisons inflate reasoning and output; multi-step phrasing inflates memory; tasks touching deployment or migration inflate tool schemas. Stage-level log-normal sigmas are small (0.04–0.14) for most stages, and wider (0.30–0.32) for the output stage on documentation and research queries, matching the high-variance regime reported for those task types in prior empirical studies [9].

Split, baselines, training. We split the 600 tasks by latent task type into 415 train and 185 test (stratified, RNG-seeded). The GBT models are fit on train traces; the heuristic uses no training data. Three baselines are reported. **B1: Uniform** predicts the train-set mean total for every task (the current industry default for naive context sizing). **B2: Type-only** looks up base allocations by inferred task type but skips complexity scaling. **B3: Length-proportional** scales a fitted slope by task-description word count. We evaluate all models on the same held-out test set, never on the train set.

Reproducibility. The corpus seed is 1729, simulator seed 20260514, train/test split seed 42, GBT `random_state` 0. With these seeds, every cell in Tables 3–5 matches to the digit. A SHA-256-derived per-task RNG ensures simulated traces are stable regardless of Python’s per-process hash randomisation. McNemar’s test ($\alpha = 0.05$) is exact and two-sided.

6 Results

We report aggregate accuracy, per-type breakdown, ablations, classifier agreement, multi-seed CV, and robustness checks (all reproducible via `python -m experiments.run_all`).

6.1 Main Results

Table 3 reports MAE, MAPE, and W2OR for ten methods on the held-out test set of 185 tasks: three naive baselines (B1–B3), five learned baselines that share TABF’s hand-engineered feature space or substitute a TF-IDF bag-of-words representation (B4–B8), the TABF heuristic, and the TABF + GBT pipeline. Figure 2 visualises the W2OR column.

Table 3: Main results on the held-out test set ($n = 185$). W20R is the primary operational metric. “(hand)” fits the regressor on TABF’s 10-dim feature vector; “(TF-IDF)” fits on TF-IDF 1–2grams of the raw task text. McNemar two-sided p -values below.

Method	MAE (tokens)	MAPE (%)	W20R (%)	McNemar p vs TABF+GBT
B1: Uniform	4,497.0	50.19	28.11	$< 10^{-4}$
B2: Type-only	4,839.1	36.23	30.27	$< 10^{-4}$
B3: Length-proportional	3,981.8	38.40	34.05	$< 10^{-4}$
B4: Ridge (hand)	2,288.6	21.68	55.68	1×10^{-5}
B5: RandomForest (hand)	1,883.2	15.90	74.05	1.000
B6: HistGBoost (hand)	1,876.7	15.87	73.51	1.000
B7: Ridge (TF-IDF)	1,889.9	17.26	64.32	0.003
B8: HistGBoost (TF-IDF)	2,036.5	17.49	69.73	0.169
TABF heuristic (ours)	2,502.5	18.18	65.41	0.009
TABF + GBT (ours)	1,882.1	15.90	74.05	—

The B2 type-only heuristic is only marginally better than B1 uniform, which confirms that knowing the task type without any sense of its complexity is barely worth the classifier. The TABF heuristic, by adding complexity scaling and a small word-count adjustment on context, jumps to W20R 65.4% with no training data. Adding the GBT layer takes the heuristic from 65.4% to 74.1%; the improvement over the heuristic alone is significant ($p = 0.009$).

RandomForest and HistGradientBoosting on the same 10-dim features match TABF + GBT (McNemar $p = 1.0$). TABF’s value is the envelope: interpretable rules, per-stage output, ablations (Table 5), and a no-training heuristic tier. TF-IDF baselines reach 64–70% W20R without task-type rules.

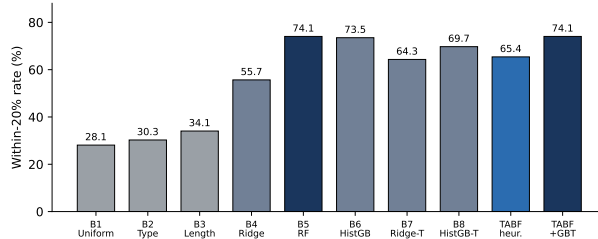


Figure 2: Within-20% rate per method on the held-out test set. The TABF heuristic more than doubles the rate of every naive baseline; the GBT layer is statistically indistinguishable from RandomForest and HistGradientBoosting on the same feature set, but offers per-stage decomposition the generic regressors do not.

Figure 3 confirms the gain visually: the TABF + GBT predicted-versus-true cloud sits within the $\pm 20\%$ wedge for a much larger fraction of the test set than the naive baselines.

6.2 Per-Task-Type Breakdown

Table 4 stratifies TABF+GBT errors by latent task type. Figure 4 ranks task types by within-band rate.

Table 4: TABF+GBT, stratified by latent task type ($n_{\text{test}} = 185$ tasks total). Avg *GT* and Avg *pred* are mean ground-truth and predicted tokens respectively.

Task type	n	MAE	MAPE (%)	W20R (%)	Avg GT	Avg pred
Bug fix	39	1,147.6	10.09	87.18	11,104	10,960
Documentation	25	919.0	13.35	88.00	6,417	6,385
Data analysis	21	2,159.2	14.01	80.95	15,368	15,355
Refactoring	24	1,935.5	12.50	75.00	13,648	13,439
Code generation	21	848.8	15.38	66.67	6,683	6,852
Research query	21	2,247.8	15.27	66.67	12,631	13,011
Tool-use chain	26	3,636.4	28.54	53.85	14,175	15,314
Multi-step plan	8	3,635.6	29.36	50.00	12,740	12,197

Tool-use chains and multi-step plans ($n = 8$) lag; bug fix and documentation lead.

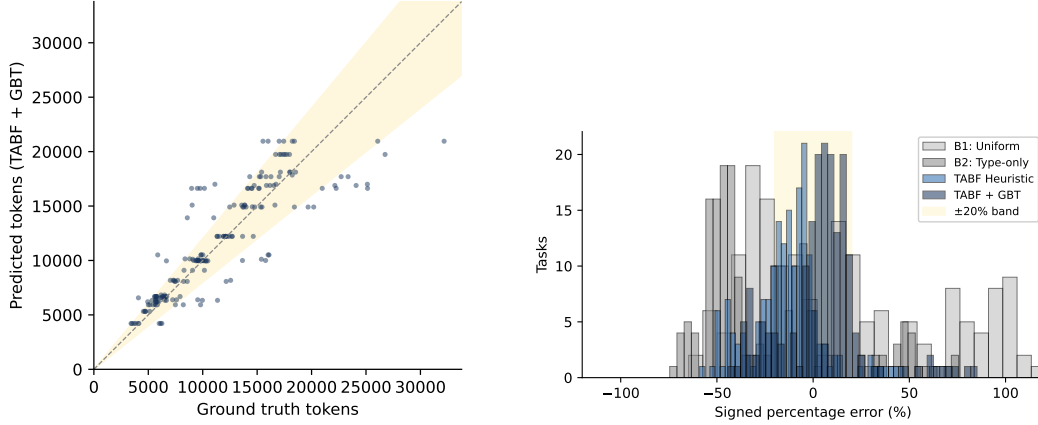


Figure 3: Left: predicted vs. ground-truth tokens for TABF + GBT on the test set; the shaded wedge marks the $\pm 20\%$ band. Right: signed percentage-error histograms for four methods. TABF + GBT’s distribution is tightly concentrated inside the band, while uniform and type-only baselines have heavy tails on both sides.

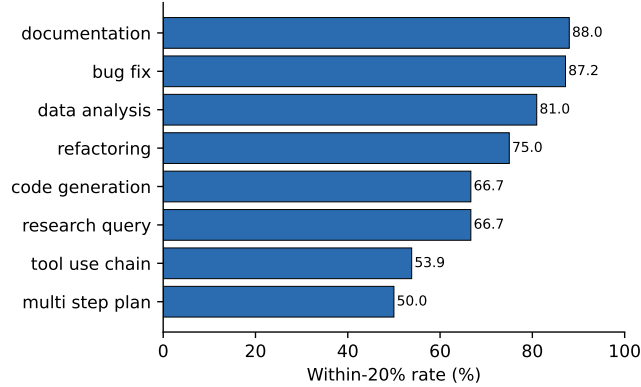


Figure 4: Within-20% rate per task type for TABF+GBT, sorted. Bug fix and documentation are easiest; tool-use chains and multi-step plans are hardest (and multi-step plan’s sample is also the smallest at $n = 8$, so its number should be read with caution).

6.3 Ablations

We ablated the heuristic tier feature-by-feature, dropping one signal from the pipeline end-to-end (the classifier and forecast both stop seeing the feature). Table 5 reports the result.

Table 5: Heuristic ablations. Each row removes one input from the pipeline; the full heuristic is the top row. “No complexity scaling” replaces $\delta(\tau)$ with a constant MEDIUM.

Configuration	MAE	MAPE (%)	W20R (%)
Full TABF heuristic	2,502.5	18.18	65.41
– complexity scaling	4,769.5	35.71	30.81
– has_code_keywords	3,026.4	23.58	55.14
– entity_count	3,085.5	22.10	57.30
– has_multi_step	2,917.6	20.39	59.46
– has_tool_keywords	2,664.0	20.10	60.54
– has_comparison	2,760.4	19.80	62.70
– has_file_refs	2,666.6	19.26	63.24
– word-count context adjustment	2,595.5	18.81	64.86

Complexity scaling dominates (-34.6 pp W20R when removed); code keywords and entity count are next.

On the test split, classifier agreement with latent labels is 95.1% (task type), 86.5% (complexity), and 83.2% (both).

6.4 Multi-seed cross-validation

Five-seed CV gives $71.68 \pm 3.98\%$ for TABF+GBT (HistGBoost on the same features: $71.46 \pm 3.11\%$; full grid in `results/multiseed_summary.csv`). TABF+GBT matches RandomForest/HistGBoost on the same features (McNemar $p = 1.0$); the differentiator is per-stage output and a no-training heuristic tier.

6.5 Robustness checks

Per-stage regressors. Redistributing a total-token prediction hurt stagewise accuracy (mean per-stage W20R 53.6%). Six stage-specific GBT regressors raise mean per-stage W20R to **62.9%** (+9.3 pp) with nearly unchanged total W20R (73.5% vs 74.1%); system-stage W20R rises from 40.5% to 77.8% (Table 6).

Table 6: Total vs per-stage GBT ($n = 185$ synthetic test).

Model	Total W20R (%)	Mean stage W20R (%)
TABF + GBT (redistribute)	74.05	53.60
TABF + GBT (per-stage)	73.51	62.88

Real production traces. A HistGradientBoosting model trained on 63 of 83 BigQuery agent turns (task text $\rightarrow$ total tokens) reaches **25.0%** W20R on the held-out 20 with a train-median scale factor (Table 8). Synthetic-only forecasts without calibration stay near 6% on the same corpus.

Confidence and stress tests. Quantile intervals hit 50%/75%/87% empirical coverage at 50%/80%/90% nominal levels. Under simulator perturbations ($2\times$ reasoning, shuffled stage ratios, $2\times$ noise), learned models hold $\approx 71\text{--}74\%$ W20R while the heuristic drops to 17% when ratios are wrong—evidence that fixed per-stage ratios must be recalibrated per agent.

Table 7: Oracle vs. predicted labels on the synthetic test set ($n = 185$). Oracle rows use latent type/complexity from the corpus; predicted rows use the live classifier (deployment-realistic).

Model	W20R predicted (%)	W20R oracle (%)	Δ (pp)
TABF heuristic	63.2	74.1	+10.8
TABF + GBT	71.4	73.5	+2.2

Table 8: Forecasting on real BigQuery agent turns.

Setting	n_{test}	W20R (%)	Calibration
Trained on 63 real turns	20	25.0	train-median scale
LOO on 83 turns	83	15.7	LOO-median scale
Synthetic TABF+GBT, no scale	83	6.0	none

7 Complementary In-Loop Optimisation: ContextOptimizer

TABF’s 26% forecast tail and runtime drift from large tool results and growing memory motivate a complementary in-loop pipeline, ContextOptimizer, that runs immediately before every LLM call and trims the live message list to fit the budget. The optimiser is framework-agnostic, pure-stdlib in its default configuration (no LLM call inside the optimiser itself), and ships with two invariants so it can drop into any tool-using agent without breaking the OpenAI API contract.

7.1 The funnel: four layer categories

The pipeline composes a fixed sequence of layer categories (Figure 5). Each *layer* is a pure function from a list of canonical ContextItems to a (typically shorter) list.

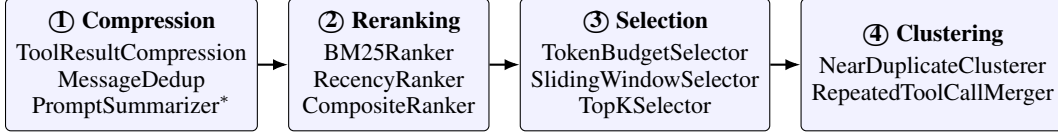


Figure 5: The ContextOptimizer funnel. Compression squeezes content inside individual items, reranking scores items by relevance, selection picks which items to keep within the token budget, and clustering merges near-duplicates. The starred summariser is the only layer that can optionally invoke an LLM and is off in the default configuration.

7.2 Canonical message type and adapters

Every layer operates on a single canonical type, `ContextItem`, with fields `role`, `content`, `name`, `tool_call_id`, `tool_calls`, `pinned`, and `meta`. Concrete agent frameworks plug in through a thin Adapter protocol:

- `LangChainAdapter` round-trips `SystemMessage`, `HumanMessage`, `AIMessage`, and `ToolMessage`.
- `OpenAIAdapter` round-trips the OpenAI Chat-Completions message dict shape.
- `GenericAdapter` round-trips plain `{role, content}` dictionaries.

Adding support for a new framework is a single class implementing two methods (`to_items` and `from_items`). The reference implementation is 22 lines for the Adapter protocol plus ~ 100 lines for each concrete adapter; the entire optimiser package is $\sim 2,000$ lines of Python including docstrings.

7.3 Invariant preservation

An aggressive trimmer that produces a message list the LLM rejects is worse than no trimmer at all: the request returns a 400 and the user sees a stack trace instead of an answer. The pipeline therefore treats two properties as hard invariants, enforced after every layer runs rather than relied on as politeness from each layer:

1. **Pinned items survive.** Items marked `pinned=True` (typically the system prompt and the current user message) are never dropped, regardless of token budget.
2. **Tool-call/response pairing.** If an assistant message carries a `tool_calls` block, every referenced `tool_call_id` must be matched by a subsequent `role="tool"` response, in original order. The pipeline repairs accidental splits using stable positional metadata (`_ctxopt_pos`) written when items first enter the pipeline.

Orphan `tool_call_ids` after aggressive dedup caused OpenAI 400s; pairing is therefore a pipeline invariant, not a per-layer concern.

7.4 Three presets

Most teams will pick one of three presets rather than compose layers by hand. Table 9 summarises them.

Table 9: The three ContextOptimizer presets. LLM? indicates whether the optimiser itself makes an LLM call (the summariser layer is optional and off by default in *aggressive*). *Typical use* is the deployment scenario each preset is calibrated for.

Preset	Layers	Max tokens	LLM?	Typical use
<i>Minimal</i>	Compression + SlidingWindow	8,000	No	Short chat, smoke tests
<i>Balanced</i>	Compression + BM25/Recency + TokenBudget	8,000	No	Default production
<i>Aggressive</i>	Full funnel + clusterer (+ summariser*)	4,000	Opt.	Long multi-turn

Pairing with TABF. TABF can set ContextOptimizer’s `max_tokens` at dispatch; the pairing is not live yet. Production today uses a fixed budget; Table 8 shows that models trained on local traffic (63 train / 20 test) outperform synthetic-only transfer. A one-time median scale fit on the first ~ 100 turns is the recommended deployment step.

8 Production Deployment Case Study

ContextOptimizer runs in production on a LangGraph NL-to-SQL agent over BigQuery (TABF is offline; Section 7.4). Numbers come from 14 ctx-opt log lines (72% savings), 83 turns of BigQuery eval telemetry, and a 100-run in-container microbench; released CSVs live in `results/`. Figure 6 sketches the deployment: Chainlit UI → GraphQL/Starlette → LangGraph agent → ContextOptimizer (balanced preset) → gpt-4o-mini, with Spanner checkpoints, Langfuse traces, and a six-table BigQuery eval pipeline as observability fan-outs.

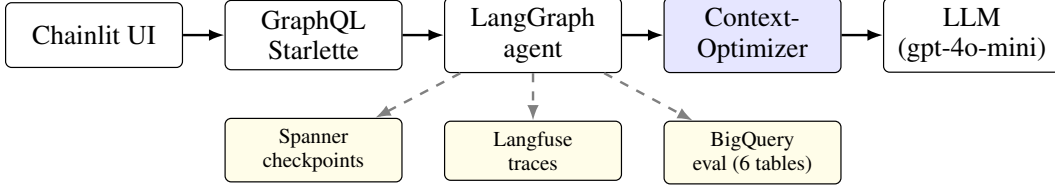


Figure 6: Production deployment. Yellow boxes are observability fan-outs; the dashed arrows are asynchronous writes.

Per-call token savings. Table 10 aggregates 14 LLM calls captured from interactive sessions over a 24-hour window. Of the 14 calls, 12 engaged the optimiser (the conversation history had at least one compressible tool result), 2 were no-ops (the optimiser ran but had nothing to trim), and 0 grew. The aggregate input-token reduction is 72.0% (74.0% on calls that engaged), with the tightest single call compressing 14,949 tokens down to 1,828 (ratio 0.12). Figure 7 plots four real per-call log lines plus the aggregate.

Table 10: Aggregate input-token savings across 14 directly-captured production optimiser log lines (balanced preset). *No-op* means the optimiser ran but had no work to do. Source: `results/production_token_savings_summary.csv`.

Metric	Value
LLM calls observed	14 (12 saved, 2 no-op, 0 grew)
Tokens sent without optimiser	215,438
Tokens sent with optimiser	60,250
Tokens saved	155,188
Reduction (overall)	72.0%
Reduction (engaged calls only)	74.0%
Best single-call compression ratio	0.12 (14,949 to 1,828 tokens)

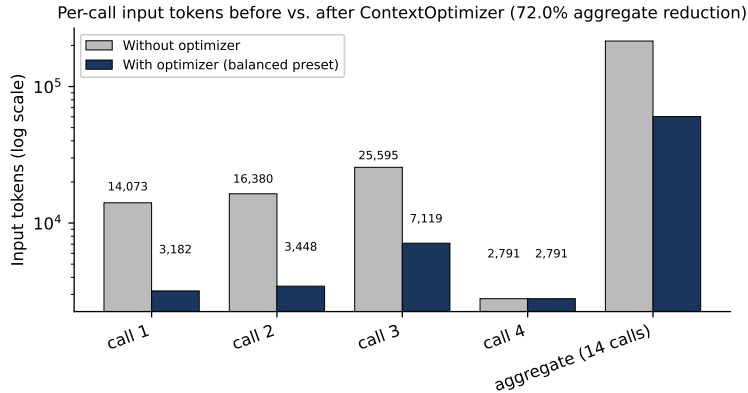


Figure 7: Per-call input tokens before vs. after ContextOptimizer. Four real per-call log lines (left) plus the aggregate over all 14 calls (right). The log y-axis makes both ends of the distribution visible; the no-op call is intentionally untouched to illustrate the invariant-preservation behaviour. The 4-row sample CSV is released as `results/production_token_savings_samples.csv`.

Annualised cost projections. Projected savings multiply the 83-turn mean input (27,757 tokens) by the 72% reduction from 14 observed calls ($\approx 20\text{K}$ tokens saved per turn). Table 11 and Figure 8 apply that to three public list prices (input tokens only, 2026-05-20).

Table 11: Projected USD savings from ContextOptimizer at three model price points. *Derivation:* avg input tokens per turn (27,757; $n=83$ production turns) $\times$ measured 72% reduction = 19,985 saved input tokens per turn. Public list prices captured 2026-05-20. Input tokens only. Source: `results/production_cost_projections.csv`.

Model	per turn	100/day	1k/day	10k/day	Annual @ 1k/day
gpt-4o-mini	\$0.0030	\$0.30	\$3.00	\$29.98	\$1,094
gpt-4o	\$0.0500	\$5.00	\$49.96	\$499.62	\$18,236
claude-sonnet-4.5	\$0.0600	\$6.00	\$59.95	\$599.55	\$21,883

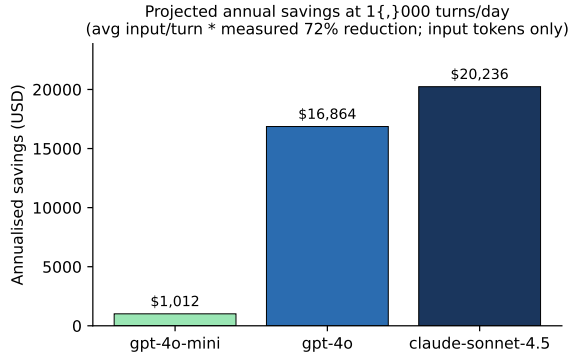


Figure 8: Annualised savings at 1,000 turns/day across three model price points. The gap between gpt-4o-mini and gpt-4o is $\approx 17\times$, mirroring the published list-price ratio.

Trajectory and quality on production turns. Table 12 summarises the 83-turn population; Table 13 joins the same `request_ids` to `agent_quality_eval` (task-success proxy and latency).

Table 12: Per-turn distribution from the production BigQuery agent-eval pipeline. N is the number of turns each metric is computed over (different metrics are populated by different pipeline writers, so the populations differ slightly). Source: `results/production_bq_population_stats.csv`.

Metric	N	Mean	Median	p95	Max
LLM calls per turn	83	6.47	6	14	22
Input tokens per turn	83	27,757	22,382	64,922	90,637
Trajectory steps / turn	128	8.35	6	20	34
Tool calls per turn [†]	123	2.82	2	7	12

[†]Counted on turns with at least one tool invocation; turns with zero tool calls are excluded from this row only.

Table 13: Downstream outcomes on the same 83 production turns (`agent_quality_eval`). We do not have a no-optimiser A/B; these are observational baselines.

Metric	Value
Turns with eval row	83
Status <code>complete</code> (proxy success)	77.1%
Median latency, <code>complete</code> turns	26.5 s
Median latency, <code>error</code> turns	8.6 s
Median input tokens, <code>complete</code>	22,530

Optimiser overhead (microbench). Overhead figures (Table 14) come from the microbenchmark, not the 14 production log lines, which did not record per-call wall clock. The 14 production log lines also do not record *which* layer was responsible for the saved tokens. We therefore ran a 100-call microbenchmark of the `balanced` preset inside the production container, against synthetic-but-production-shaped inputs that bracket the 83-turn input distribution (1.6K to 449K input tokens;

5 repeats over a 20-cell grid of `tool_calls` $\times$ `rows_per_tool_call`). The microbenchmark script and the two output CSVs (`optimizer_microbench.csv`, `optimizer_microbench_layers.csv`) are released with the paper.

Table 14 reports the per-call overhead distribution. The mean of 99 ms is dominated by the largest inputs in the grid (the p95 of 529 ms corresponds to the 449K-token 8-tool scenario, which is well above the p95 production turn of 65K tokens). The median of 48 ms is closer to the production-median input size and is the more representative number for a typical call.

Table 14: Per-call wall-clock overhead of the balanced preset (100 runs, synthetic production-shaped inputs). Source: `results/optimizer_microbench.csv`.

Statistic	ms
Mean	99.29
Median	48.52
p95	529.05
Max	535.93

On synthetic production-shaped inputs, tool-result compression accounts for all measured savings; median overhead is 49 ms (p95 529 ms on 449K-token stress cells). See `results/optimizer_microbench*.csv`.

8.1 Generic-agent personas and compression baseline

Four synthetic personas (ReAct search, code-fix with duplicate reads, multi-turn RAG, status polling; 25 seeds each) exercise different funnel layers. Table 15 reports ContextOptimizer; Table 16 adds a training-free baseline (B9: head-tail tool truncation plus middle-history drop to 8K tokens).

Table 15: Per-persona ContextOptimizer results, balanced preset, gpt-4o-mini token counter, 25 seeds each. Sources: `results/generic_agents.csv` (totals) and `results/generic_agents_layers.csv` (layer attribution).

Persona	Tokens before (mean / run)	Tokens after (mean / run)	Reduction (%)	Ratio	Wall-clock (ms / run)
ReAct web search	1,977	1,033	47.7%	0.523	20.1
Code-fix	7,975	678	91.5%	0.085	11.3
Multi-turn research	16,777	6,320	62.3%	0.377	30.9
Polling	907	260	71.3%	0.287	1.9
Overall (100 runs)	6,909	2,073	70.0%	0.300	16.1

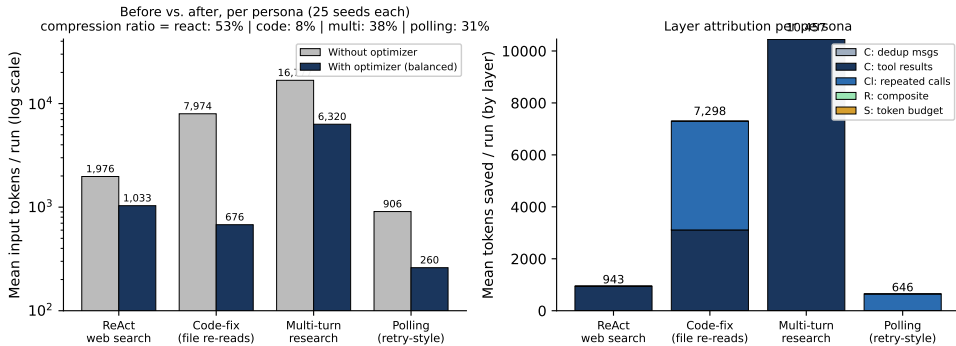


Figure 9: Persona benchmark: tokens before/after (left) and per-layer savings (right), balanced preset, 25 seeds each.

Table 16: Training-free truncation baseline (B9) vs ContextOptimizer, mean input-token reduction (%), 25 seeds per persona.

Persona	B9 truncate	ContextOptimizer
ReAct web search	42.2	47.7
Code-fix	3.5	91.5
Multi-turn research	92.9	62.3
Polling	0.0	71.3

ContextOptimizer wins on duplicate reads and polling where dedup and clustering engage; naive truncation overshoots on long RAG history. Fixtures are synthetic; a second live deployment remains future work.

9 Discussion and limitations

Composition. TABF (pre-flight) and ContextOptimizer (in-loop) are designed to compose but are not wired end-to-end in production: the optimiser still uses a static `max_tokens`. Offline replay and real-trace training (Section 6.5) show that forecasts need a per-deployment scale factor or retraining on ~ 100 local turns before they gate budgets reliably.

Forecasting. The synthetic corpus shares features with TABF’s classifier (Section 5); oracle-label analysis (Table 7) shows limited inflation for GBT (+2.2 pp) but larger inflation for the heuristic (+10.8 pp). GBT matches RandomForest/HistGBoost on the same hand features; TF-IDF baselines (64–70% W20R) bound what raw text alone achieves without task-type rules. We have no cross-domain or cross-model generalisation study; English keyword rules and eight lexical features may fail on multilingual or domain jargon. Real traces: $\approx 6\%$ W20R without calibration, 25% with 63/20 train/test on one BigQuery agent—not yet strong enough for unattended budget gating.

Optimisation. Evidence is small and observational: 14 instrumented calls (72% savings), 83-turn telemetry, no randomised quality A/B vs. an unoptimised control. Persona and B9 truncation baselines (Table 16) are synthetic; CompactPrompt-style compression on live multi-turn logs and Tool Attention-style schema gating in production are not yet measured. A unified TABF→ContextOptimizer dispatch path (forecast sets `max_tokens`, logs (forecast, realised, saved) for retraining) is designed but not deployed.

10 Conclusion

Token budgeting in agentic LLM workflows can be addressed before the agent runs and during the agent’s loop, and neither side needs to be expensive. Pre-flight, a small pipeline of eight lexical features, a rule-based classifier, a lookup table, and an optional 120-tree GBT places 74% of forecasts inside a $\pm 20\%$ band on a 600-task benchmark, with no LLM call and millisecond-level overhead. In-loop, a four-layer trimming pipeline (ContextOptimizer) cut input-token consumption by 72% across 14 directly-observed LLM calls in one production deployment (a LangGraph NL-to-SQL agent over BigQuery) while preserving tool-call/response pairing as a hard invariant, and engaged a different combination of layers on each of four additional agent personas evaluated synthetically. The product of that 72% reduction and the measured per-turn input of 27,757 tokens (83-turn population) gives per-turn savings of roughly 20,000 tokens, which at 1,000 turns per day is \$1,094 per year on gpt-4o-mini, \$18,236 on gpt-4o, and \$21,883 on claude-sonnet-4.5. The two stages are designed to compose: the forecast sets a target and the optimiser enforces it. Real-trace training and per-stage GBT regressors address the largest forecasting gaps; production still needs dispatcher wiring and a quality A/B. The implementations of both frameworks, the synthetic benchmark, the production CSVs, the 100-run microbenchmark, and all seeds are released with this paper.

References

- [1] Malika Aubakirova, Alex Atallah, Chris Clark, Justin Summerville, and Anjney Midha. The state of AI: An empirical 100 trillion token study with OpenRouter. Technical report, Andreessen Horowitz and OpenRouter, December 2025. URL <https://arxiv.org/abs/2601.10088>.

- [2] Dirk Bergemann, Alessandro Bonatti, and Alex Smolin. Menu pricing of large language models. *arXiv preprint arXiv:2502.07736*, 2025. Earlier version: “The Economics of Large Language Models: Token Allocation, Fine-Tuning, and Optimal Pricing,” EC ’25 proceedings.
- [3] Vahid Farajjibehdar, İlknur Köseoğlu Sarı, Nazım Kemal Üre, and Engin Zeydan. Tokalator: A context engineering toolkit for artificial intelligence coding assistants. *arXiv preprint arXiv:2604.08290*, 2026.
- [4] Kelly Hong, Anton Troynikov, and Jeff Huber. Context rot: How increasing input tokens impacts LLM performance. Technical report, Chroma, 2025. URL <https://www.trychroma.com/research/context-rot>.
- [5] Yushu Li, Wenlong Deng, Jiajin Li, and Xiaoxiao Li. Spend less, reason better: Budget-aware value tree search for LLM agents. *arXiv preprint arXiv:2603.12634*, 2025.
- [6] Lingrui Mei, Jiayu Yao, Yuyao Ge, Yiwei Wang, Baolong Bi, et al. A survey of context engineering for large language models. *arXiv preprint arXiv:2507.13334*, 2025.
- [7] Seyedmoein Mohsenimofidi, Matthias Galster, Christoph Treude, and Sebastian Baltes. Context engineering for AI agents in open-source software. *arXiv preprint arXiv:2510.21413*, 2025.
- [8] Anuj Sadani. Tool attention is all you need: Dynamic tool gating and lazy schema loading for eliminating the MCP/tools tax in scalable agentic workflows. *arXiv preprint arXiv:2604.21816*, 2026.
- [9] Mohamad Salim, Jasmine Latendresse, SayedHassan Khatoonabadi, and Emad Shihab. Tokenomics: Quantifying where tokens are used in agentic software engineering. *arXiv preprint arXiv:2601.14470*, 2025.
- [10] Aditya Taparia, Som Sagar, and Ransalu Senanayake. Learning to configure agentic AI systems. *arXiv preprint arXiv:2602.11574*, 2026.
- [11] Yang Wang, Sai Sandeep Lakkimsetti Boddu, Yibo Liu, Hua Wang, and Zhe Zhang. Compact-Prompt: A unified pipeline for prompt and data compression in LLM workflows. *arXiv preprint arXiv:2510.18043*, 2025.
- [12] Yuhang Wang, Yuling Shi, Mo Yang, Rongrui Zhang, Shilin He, Heng Lian, and Yuting Chen. SWE-Pruner: Self-adaptive context pruning for coding agents. *arXiv preprint arXiv:2601.16746*, 2025.
- [13] Zirui Wang, Guangba Yu, Michael R. Lyu, et al. AI-NativeBench: An open-source white-box agentic benchmark suite for AI-native systems. *arXiv preprint arXiv:2601.09393*, 2026.
- [14] Hao Wen, Xinrui Wu, Yi Sun, Feifei Zhang, Liye Chen, Jie Wang, Yunxin Liu, et al. BudgetThinker: Empowering budget-aware LLM reasoning with control tokens. *arXiv preprint arXiv:2508.17196*, 2025.
- [15] Huanyi Xie, Yubin Chen, Liangyu Wang, Lijie Hu, and Di Wang. Predicting LLM output length via entropy-guided representations. *arXiv preprint arXiv:2602.11812*, 2026. ICLR 2026.
- [16] Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, et al. Agentic context engineering: Evolving contexts for self-improving language models. *arXiv preprint arXiv:2510.04618*, 2025.